

RL Inventory Control

Leaderboard Submission Guide & Technical Interface Contract

COURSE ID

ID6002W

1. Access & Authentication

Follow these quick steps to access the environment evaluation system and set up your profile profile:

<LEADERBOARD_URL>

- Click "**Login with IITM Email**" and sign in using your official address (@smail.iitm.ac.in or @iitm.ac.in).
- Your **roll number is derived automatically** from your email prefix (e.g., da24s016@smail.iitm.ac.in → roll DA24S016).

Important Roll Number Alignment:

Ensure you log in with the email matching your official roll number. Server evaluation instances are programmatically generated from your exact roll number prefix. Logging in with a mismatched account will evaluate your policy on the wrong environment variant.

2. Submission Package Structure

You submit **one trained policy (one RL technique) per submission**. To climb the leaderboard, submit evaluations across multiple distinct techniques.

- Select your algorithm from the technique dropdown (see §6).
- Upload either a single `policy.py` file **or** a `.zip` archive containing `policy.py` at its root level.
- **Max upload limit:** 50 MB.

Correct Root Zipping: `zip submission.zip policy.py model.pt`

Incorrect Directory Zipping: `zip submission.zip myfolder/policy.py`

3. Policy Interface Contract

Your `policy.py` file must define the entry point with this exact function signature:

```
def run_policy(observation) -> list | tuple | np.ndarray:
    ...
    return [a1, a2, a3]  # exactly 3 actions
```

Observation Dictionary Schema

KEY	SHAPE	DTYPE	MEANING
inventory	(3,)	int32	Current on-hand stock for the 3 products
arrival_pipeline	(3, 4)	int32	Units ordered, arriving over the next 4 days
demand_history	(7, 3)	int32	Last 7 days of demand per product
day	(1,)	int32	Current day index (0-based)
capacity_utilisation	(1,)	float32	Warehouse capacity used (0.0 – 1.0)

Action Output Conventions

Return **exactly 3 values** (one per product). The server supports both output conventions and auto-detects your output style:

- **Action Indices (0–10):** Represents order quantity = index × 10 (e.g., `[0, 10, 5]` → 0, 100, 50 units).
- **Order Quantities:** Direct multiples of 10 up to 100 (e.g., `[0, 100, 50]`).

```

import numpy as np

def run_policy(observation):
    inv = observation["inventory"]          # np.int32 array of length 3
    actions = []
    for i in range(3):
        # Order 60 units (index 6) when stock is low, else 0
        actions.append(6 if inv[i] < 50 else 0)
    return actions

```

4. Local Evaluation Harness

To ensure your submission succeeds on the server, test locally using the course env package:

```

import numpy as np
from env import make_env
from policy import run_policy

roll = "DA245016"  # <-- Replace with your official roll number
costs = []

for ep in range(20):
    seed = 1000 + ep          # Server uses seeds 1000..1019
    env = make_env(roll, scenario_mode="random", domain_randomization=True)
    obs, _ = env.reset(seed=seed)
    done = False
    ep_cost = 0.0

    while not done:
        act = np.asarray(run_policy(obs), dtype=np.int64)
        if np.any(act > 10):    # Server auto-conversion logic
            act = env.quantities_to_action_indices(act)
        obs, reward, terminated, truncated, info = env.step(act)
        ep_cost += info["costs"]["daily_total"]
        done = terminated or truncated
    costs.append(ep_cost)

print("Avg public cost over 20 eps:", round(sum(costs)/len(costs), 2))

```

5. Benchmark Architecture & Allowed Algorithms

Leaderboard standing is based on minimizing average operational costs (holding + stockout + ordering + discarding). Ranking requires submitting at least 5 distinct techniques from the approved collection below:

1. Tabular Q-Learning

2. Tabular SARSA

3. TD(λ) Eligibility Traces

4. NN-based Q-Learning

5. NN-based SARSA

6. REINFORCE (+/- Baseline)

7. Advantage Actor-Critic (A2C)

8. Asynchronous A2C (A3C)

9. Proximal Policy Opt (PPO)

10. Deep Q-Network (DQN)

11. Double DQN

6. Execution Requirements & Project Deliverables

CONSTRAINT 01

60-Second Execution Budget

Your submission must complete 20 evaluation episodes (1,000 steps) within 60 seconds total. Pre-train network weights offline and load them at import time in `policy.py`.

CONSTRAINT 02

Server Runtime Environment

Pre-installed packages include `numpy`, `gymnasium`, `torch`, and `stable-baselines3`. Standard output (`print` statements) is suppressed during evaluation.

Project Reports & Notebooks

Upload these required items via **My Dashboard** prior to the final course deadline:

- **Jupyter Notebook** (.ipynb): Reproducible training and validation codebase.
- **Project Report** (.pdf): Summary document strictly limited to a maximum of 4 pages.

7. Troubleshooting Guide

DASHBOARD ERROR MESSAGE	CAUSE & REMEDIATION
policy.py not found in root...	Zip file contains nested directory structures. Re-zip strictly at the root level.
run_policy(...) function missing	Ensure run_policy is defined at the top-level scope of policy.py.
...must return a sequence of 3...	Return exactly 3 numeric action outputs per function call.
Import/Execution Error	Syntax error, missing module, or missing asset file. Test execution locally.
Runtime Evaluation Error	Runtime exception encountered inside the step loop (e.g., array index out-of-bounds).
Execution Timed Out (>60s)	Exceeded evaluation budget. Pre-train models offline instead of inside the evaluation loop.